

P講習 Flaskアプリ開発

22NC051 藤原滉人
22FI103 本間航介
22FI116 山岸愛翔

はじめに

- この講習ではflaskというWebアプリケーションフレームワークを使ってTODOアプリをハンズオンで作成します
- アプリ作成を通して、アプリ開発の基礎となるCRUD機能の実装方法やDBの使い方を学びます
- 今後のグループ開発(P講習)でflaskを活用します. この授業の中でflaskを使えるようにしましょう
- 作業はvscodeを利用します

Flaskとは

- Flask(フラスク)はPythonのWebアプリケーションフレームワークの1つ
- アプリ作成に必要な機能が提供されており, 機能を利用することで簡単にアプリ開発を行うことができる
- 他フレームワークと比べ, シンプルで軽量であり学習コストが低い

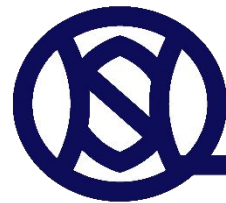


Flask



今回作るもの

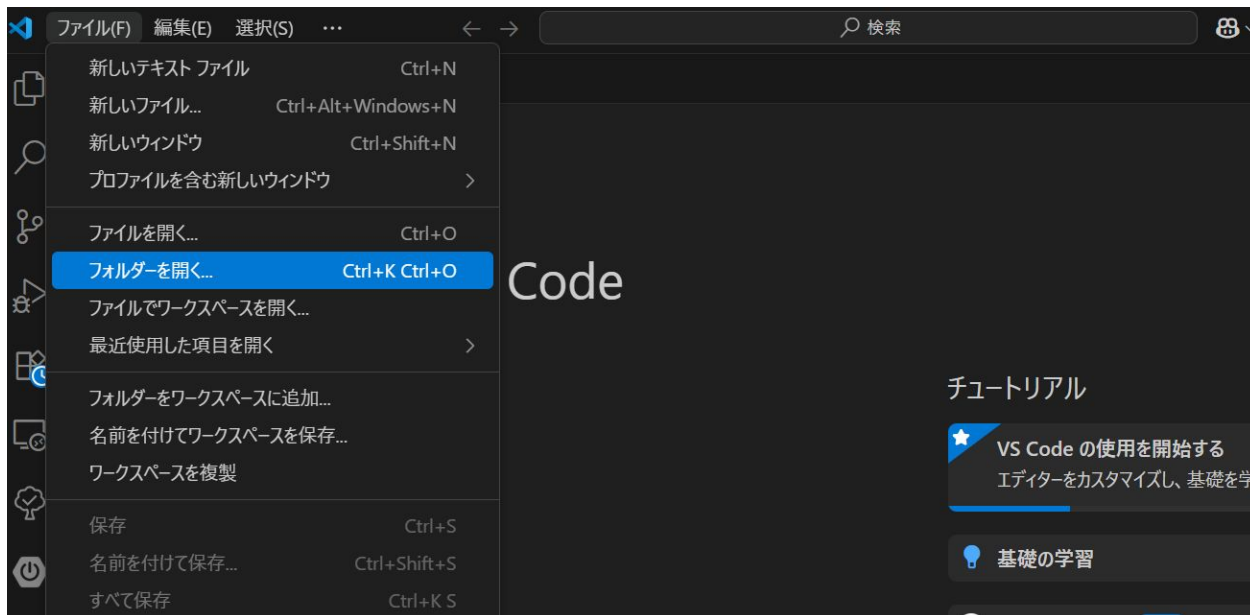
- TODOアプリ
 - タスク作成機能(Create)
 - タスク表示機能(Read)
 - タスク編集機能(Update)
 - タスク削除機能>Delete)



準備

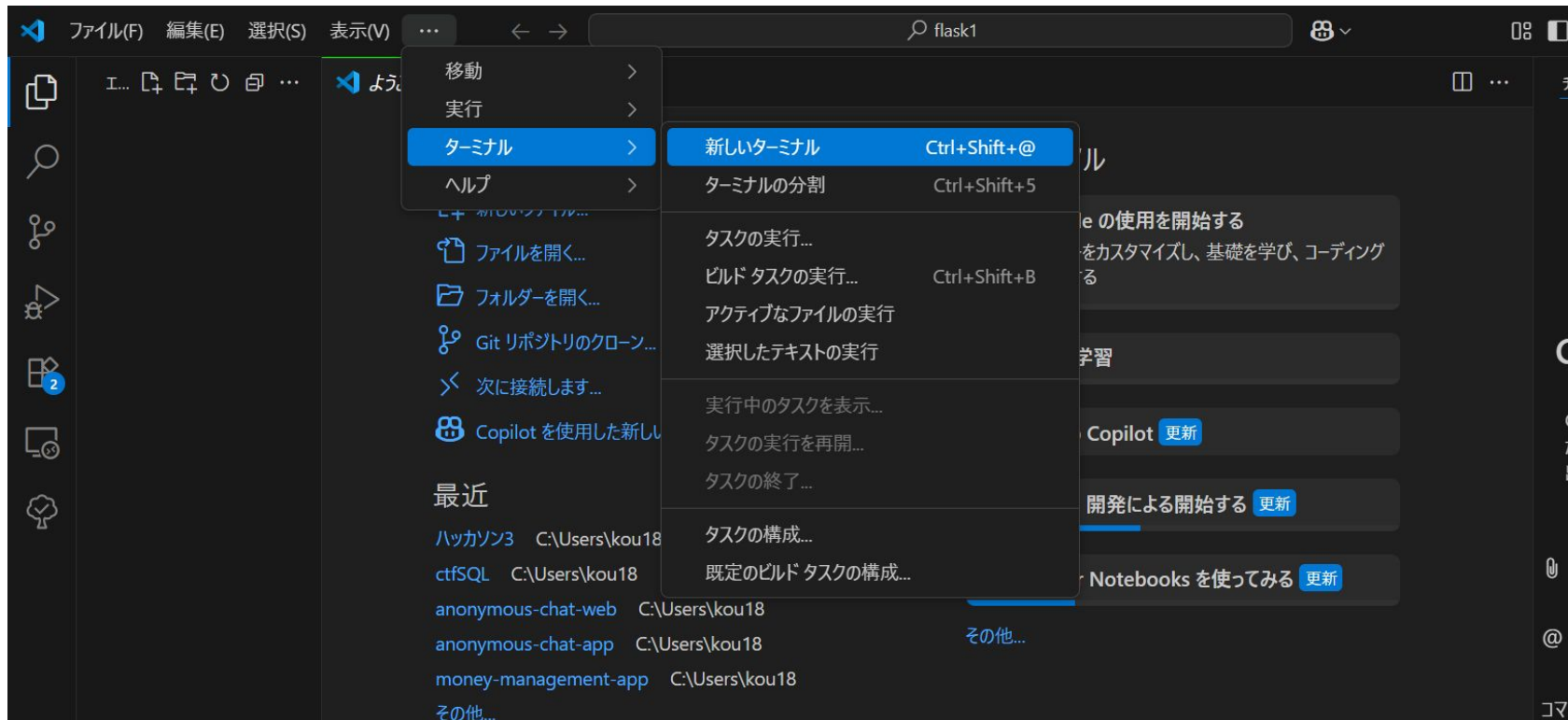
作業フォルダの作成

1. 任意の場所に「psemi-flask」という作業フォルダを作成します
2. vscodeで作成したフォルダを開きます



ターミナルの出し方

- 以下の画像を参考に新しいターミナルを表示する



仮想環境の作成とアクティベート

1. 仮想環境を作成します. 以下をターミナルに入力

```
python -m venv venv
```

1. アクティベートします

```
venv¥Scripts¥activate
```



補足: 仮想環境とは

仮想環境については下記資料の「環境を分けると何が嬉しいの？」がわかりやすいので不安な場合参考にしてください.

<https://zenn.dev/nekoallergy/articles/py-env-pipenv01>

必要なライブラリのインストール

今回必要なライブラリは以下の2つです.

- Flask
もちろん必須
- Flask-SQLAlchemy
データベースの操作に必要(詳細後述)

以下コマンドでインストールしましょう

```
pip install Flask
```

```
pip install Flask-SQLAlchemy
```

requirements.txt

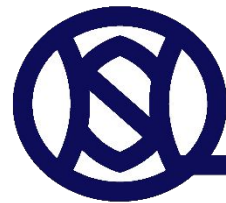
作業環境にインストールしたライブラリを管理するファイル
これにより常に同じ環境を作成することができる

- requirements.txtの生成

```
pip freeze > requirements.txt
```

(requirements.txtを使ったライブラリのインストール)

```
pip install -r requirements.txt
```



アプリ基礎

Hello worldの表示

psemi-flask配下にapp.pyを作成し、以下のソースコードを記載します

※インデントを下げる(Tabキーを押す)

returnの行
はTabキー
を押す！

この行も
Tabキーを
押す！

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def index():
    return "Hello, World!"

if __name__ == "__main__":
    app.run(debug=True)
```

Hello worldの表示

以下のコマンドで実行します

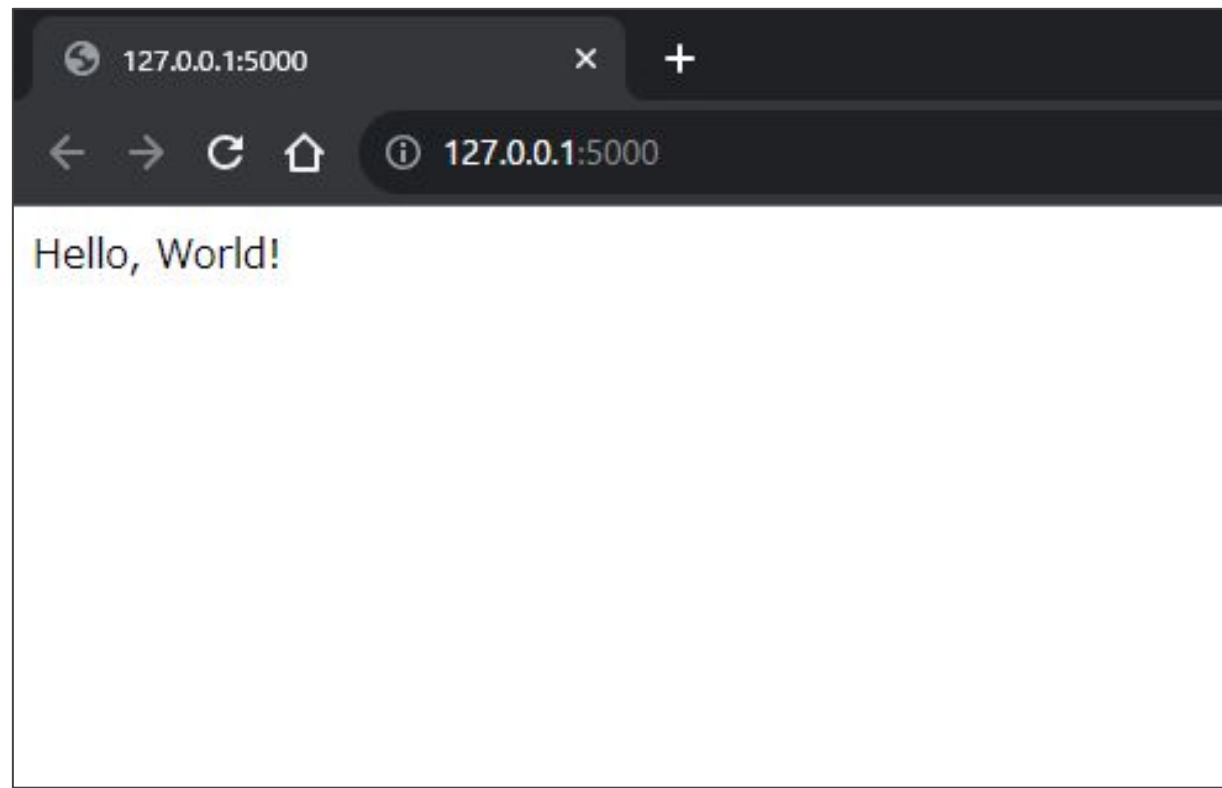
```
python app.py
```

画像のようになったら, ブラウザで <http://127.0.0.1:5000> にアクセスしてみましょう

```
(venv) PS C:\psemi\psemi-flask> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 118-791-137
```

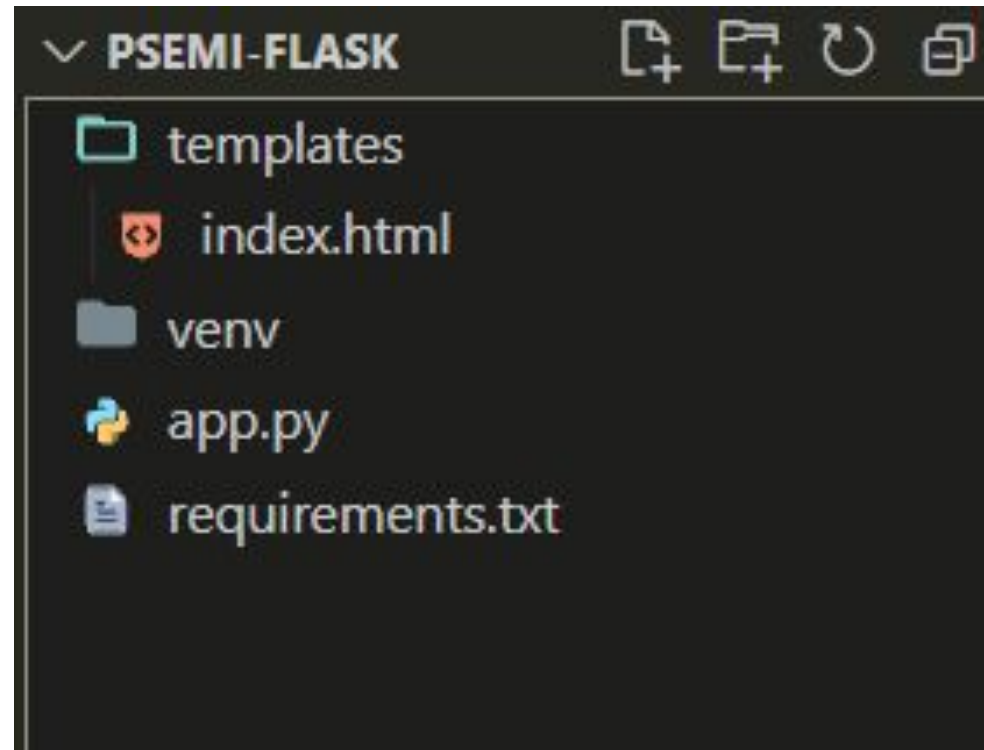
Hello worldの表示

「Hello, World!」が表示されていれば成功です



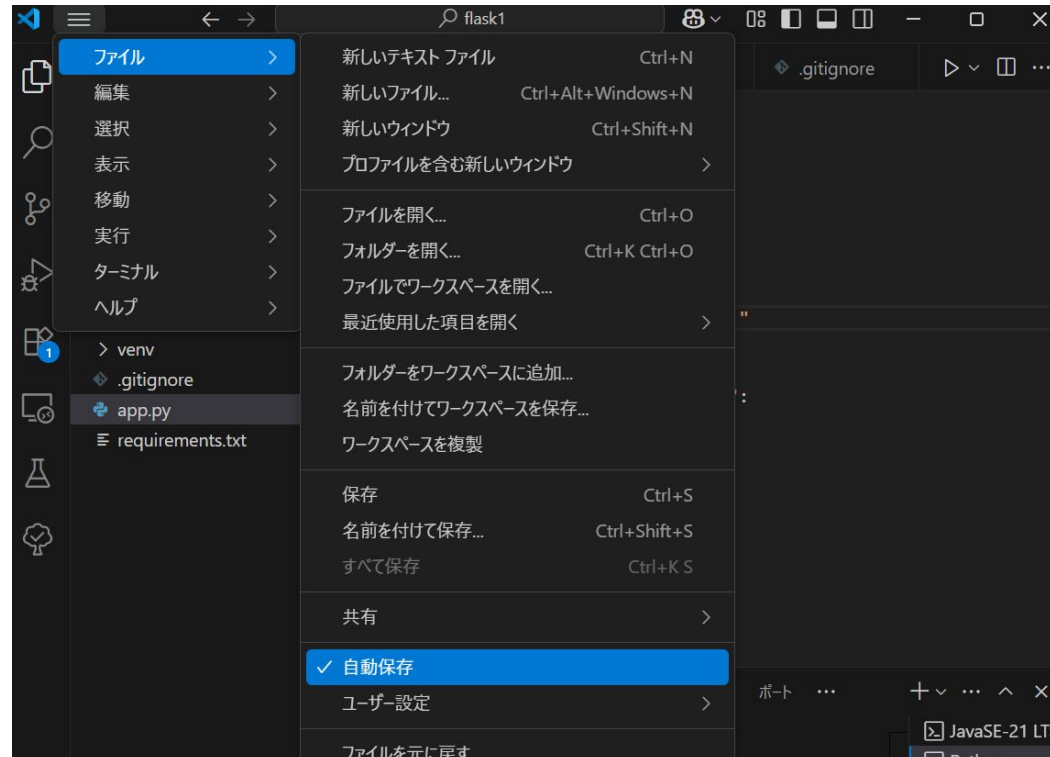
HTMLの表示

新たにtemplatesフォルダを作成し、配下にindex.htmlを作成します
templatesフォルダはhtmlファイルを格納するためにフォルダです



自動保存

- ファイルから自動保存を押す(✓マークがあればOK)
 - 実行するpythonファイル以外の変更は実行ごとに保存する必要があるが、自動保存にすることでその必要がなくなる



HTMLの表示

index.htmlには以下の内容を記載します

```
<!DOCTYPE html>
<html lang="ja">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
  </head>

  <body>
    <h1>Hello World</h1>
  </body>
</html>
```

HTMLの表示

app.pyを以下のように変更します

```
from flask import Flask, render_template # 追加

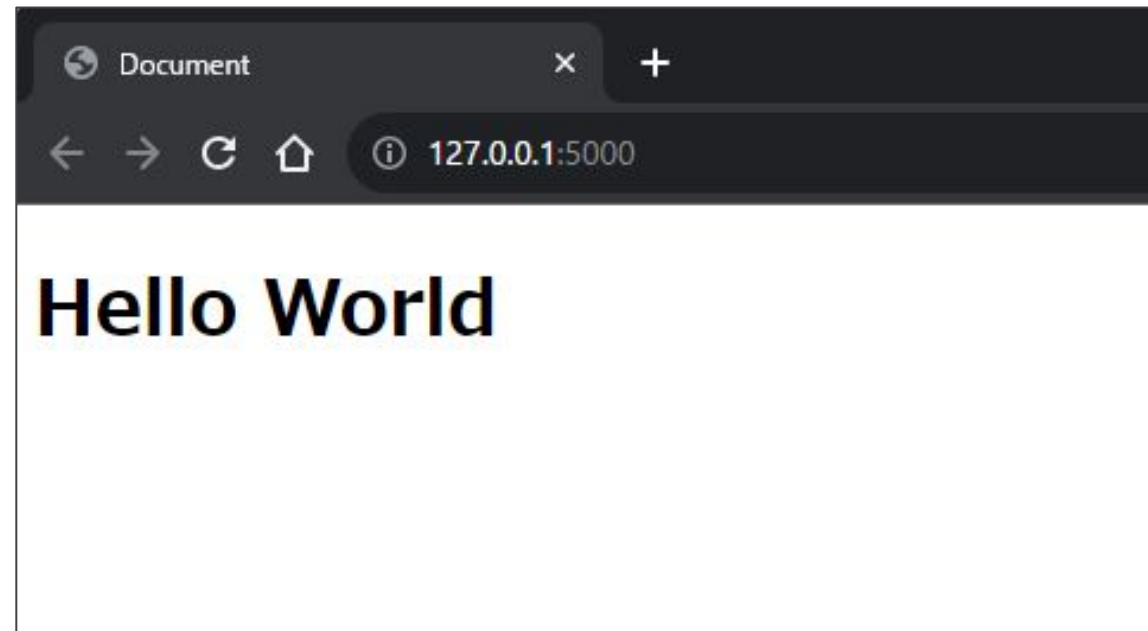
app = Flask(__name__)

@app.route("/")
def index():
    return render_template("index.html" ) # 変更

if __name__ == "__main__":
    app.run(debug=True)
```

HTMLの表示

画像のように文字が大きく表示されていれば成功です



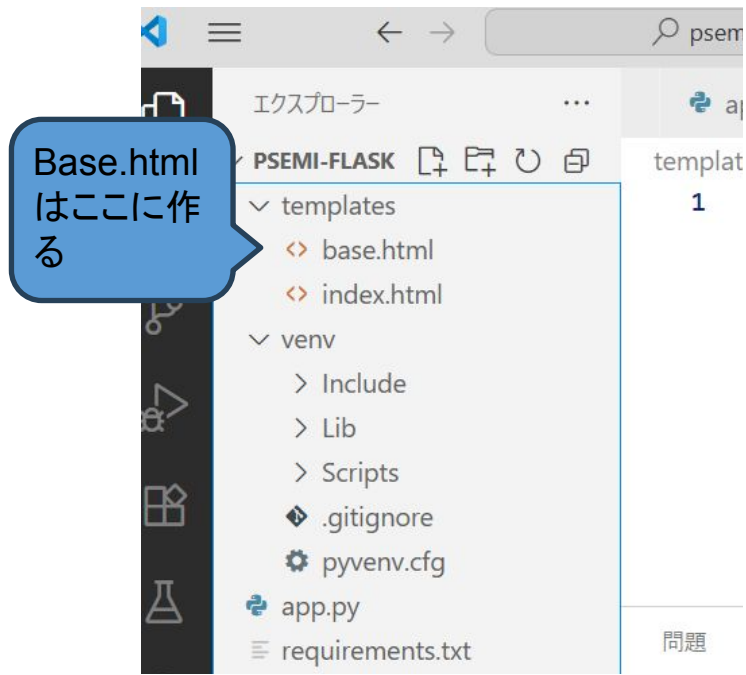
補足:HTMLタグ

以下のURLにHTMLタグの一覧が記載されているため、参考にしてください。

<https://service.biztex.co.jp/dx-hacker/it-dx-column/htmltag/>

共通テンプレート

基礎になるHTMLファイル(base.html)をtemplatesフォルダの配下に用意することでそのファイルを他のHTMLファイル(index.html)でも使いまわせる



```
<!DOCTYPE html>
<html lang="ja">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Todo app</title>
    {% block head %}{% endblock %}
  </head>

  <body>
    {% block body %}{% endblock %}
  </body>
</html>
```

共通テンプレート

共通テンプレートによってindex.htmlはこれだけ

```
{% extends 'base.html' %}
```

```
{% block body %}
```

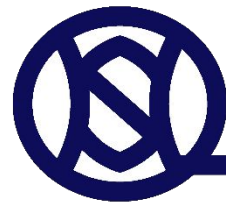
```
<h1>トップページ</h1>
```

```
{% endblock %}
```

共通テンプレート

下の画像のように表示されれば成功





DBの準備

DBについて

データベースには様々な種類があり、用途によって使い分ける
例:PostgreSQLやSQLite(リレーショナル型)、MongoDB(NoSQL)など

今回は**SQLite**を使用する

SQLAlchemy

Pythonでよく利用されるORMの1つ

データベースをオブジェクトとして扱い、クエリを関数で書ける

例) DBから特定のユーザー情報を取得するとき

SQL

```
"SELECT id, name, email, country FROM users WHERE id = 20"
```

ORM

```
User.query.get(id = 20)
```

データベースの準備

app.pyを以下のようにします

追加！

```
from flask import Flask, render_template
from flask_sqlalchemy import SQLAlchemy # 追加
```

追加！

```
app = Flask(__name__)
# 以下追加
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///todo.db"
db = SQLAlchemy(app)

class Todo(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(100))
    details = db.Column(db.String(100))

@app.route("/")
def index():
    return render_template("index.html")

if __name__ == "__main__":
    app.run(debug=True)
```

解説: データベースの準備

使用するデータベースの場所を指定

dbインスタンスを作成

dbインスタンスはDBの操作に利用

(これは先ほど追加したコードの説明！)

```
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///todo.db"  
db = SQLAlchemy(app)
```

解説: データベースの準備

Todoという名前のデータベースモデルを定義

- id:int型, 主キー(一意に識別するためのもの)
- title:str型, 最大100文字
- details:str型, 最大100文字

```
class Todo(db.Model):  
    id = db.Column(db.Integer, primary_key=True)  
    title = db.Column(db.String(100))  
    details = db.Column(db.String(100))
```

todo.dbの作成

エラーが出ると思うので次ページも参照)

以下を入力し, Pythonの対話モードに入る

```
python
```

以下を入力し, DBのテーブルを作成する

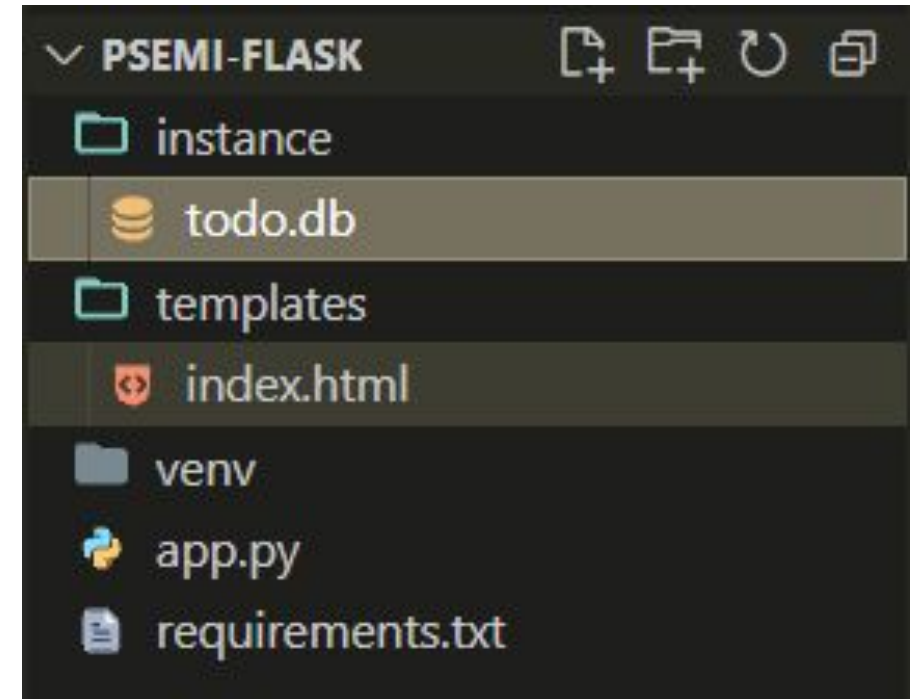
```
>>>from app import app, db
```

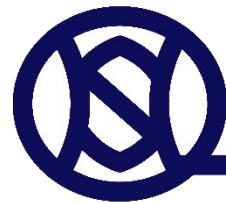
```
>>>with app.app_context():
```

```
...     db.create_all()
```

半角スペース4マス

※exit() またはquit()で対話モードを終了できる





各機能の実装

準備

機能実装の前に, app.pyに新たにrequestとredirectを追加します

```
from flask import Flask, render_template, request, redirect # 変更  
from flask_sqlalchemy import SQLAlchemy
```

タスク作成

タスクを作成するためのフォームを作ります
index.htmlに以下を追加してください

追加！

```
{% block body %}  
<h1>トップページ</h1>  
<div class="form">  
  <form action="/create" method="POST">  
    <label for="title">Title</label>  
    <input type="text" name="title" />  
    <label for="details">Details</label>  
    <input type="text" name="details" />  
    <input type="submit" value="Create" />  
  </form>  
</div>  
{% endblock %}
```

GETとPOST

アプリ開発においては情報を送信するときには、
POSTメソッドを利用します
情報を取得するときはGETメソッドを利用します

詳細は以下資料を参考にしてください。

<https://qiita.com/kanataxa/items/522efb74421255f0e0a1>

タスク作成

フォームに入力された内容は, /createにPOSTメソッドで届きます
create関数には"/create"に届いたリクエスト(フォーム内容)を処理し
タスクをDBに保存する処理を書きます

```
# 以下追加
@app.route("/create", methods=["POST"])
def create():
    title = request.form.get("title")
    details = request.form.get("details")
    new_task = Todo(title=title, details=details)
    db.session.add(new_task)
    db.session.commit()
    return redirect("/")
```

解説: タスク作成

リクエストは `request.form.get` でアクセスできます
`title`はフォームのinputタグの`name`属性です
これをもとに, `new_task`インスタンスを作成します

```
title = request.form.get("title")  
details = request.form.get("details")  
new_task = Todo(title=title, details=details)
```

解説: タスク作成

db.session.add()でDBに新規オブジェクトを追加することができます

db.session.commit()で追加内容を保存します

タスクが保存出来たら, "/"に戻るようにします

```
db.session.add(new_task)
db.session.commit()
return redirect("/")
```

タスクの表示

タスクを表示するために, index関数を以下のように変更します
Todo.query.all()でDBから全てのタスクを取得できます
tasksはTodoクラスのオブジェクトをもつ配列です
このtasksをindex.htmlに渡します

```
@app.route("/")  
def index(): # 以下変更  
    tasks = Todo.query.all()  
    return render_template("index.html", tasks=tasks)
```

app.py

タスクの表示

index.htmlにタスクを表示するための処理を書きます

追加！

```
{% extends 'base.html' %}
{% block body %}

<h1>トップページ</h1>
<div class="form">
  <form action="/create" method="POST">
    <label for="title">Title</label>
    <input type="text" name="title" />
    <label for="details">Details</label>
    <input type="text" name="details" />
    <input type="submit" value="Create" />
  </form>
</div>

{% for task in tasks %}
  <div class="task">
    <h2>タイトル : {{ task.title }}</h2>
    <p>詳細 : {{ task.details }}</p>
  </div>
{% endfor %}
{% endblock %}
```


タスク作成, 表示

タスクが作成でき, 表示されることを確認しましょう



Todo app

127.0.0.1:5000

トップページ

Title Details



Todo app

127.0.0.1:5000

トップページ

Title Details

タイトル : 研究

詳細 : 論文調査

タスクの削除

タスク削除機能を実装します

index.htmlの各タスクに削除のボタンを追加しましょう

```
{% for task in tasks %}  
  <div class="task">  
    <h2>タイトル : {{ task.title }}</h2>  
    <p>詳細 : {{ task.details }}</p>  
    <a href="/delete/{{ task.id }}" role="button">Delete</a> # 1行追加  
  </div>  
{% endfor %}  
{% endblock %}
```

タスクの削除

削除ボタンを押すと”/delete/id”にリクエストが届きます

Todo.query.get()で特定のタスクを取得することができます

db.session.delete()でDBからデータを削除できます

```
@app.route('/delete/<int:id>')
def delete(id):
    delete_task = Todo.query.get(id)
    db.session.delete(delete_task)
    db.session.commit()
    return redirect('/')
```

app.py

タスクの削除

Deleteをクリックすると、タスクが削除できるか確認しましょう



Todo app

127.0.0.1:5000

トップページ

Title Details

タイトル：研究

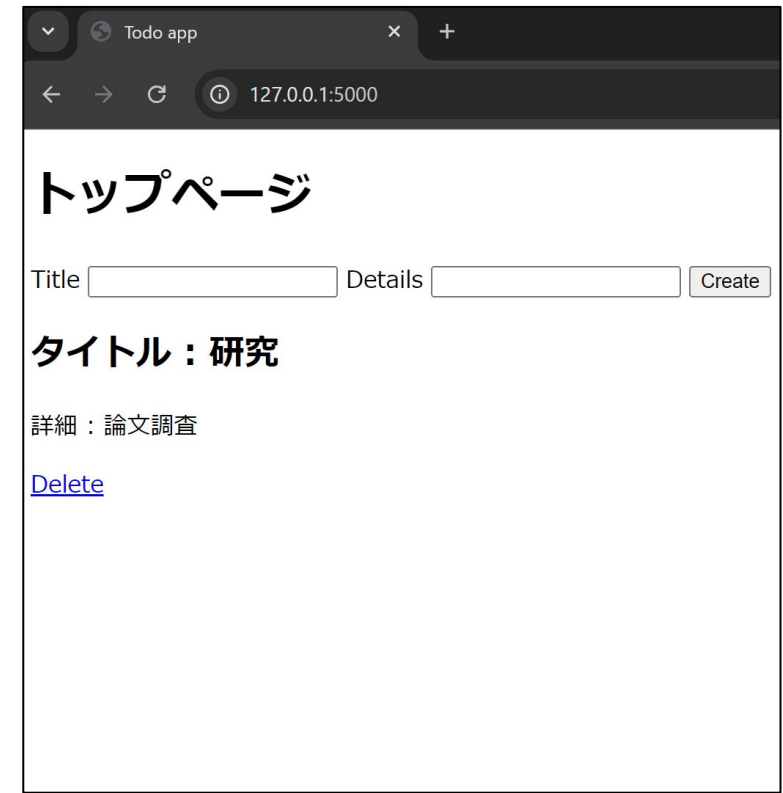
詳細：論文調査

[Delete](#)

タイトル：ああああああああ！

詳細：いいいいいい

[Delete](#)



Todo app

127.0.0.1:5000

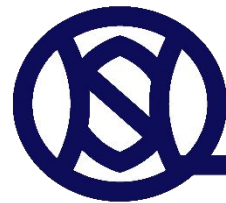
トップページ

Title Details

タイトル：研究

詳細：論文調査

[Delete](#)



課題

必須課題

1. タスクの編集機能(update)の実装しよう

2. CSSを利用できるようにしよう

タイトル: P講習課題

詳細: 1. タスクの編集機能(update)の実装しよう

[Update](#) [Delete](#)

課題1の例

トップページ

Title

Details

Create

タイトル: P講習課題

詳細: 2. CSSを利用できるようにしよう

[Update](#)

[Delete](#)

タイトル: 筋トレ

詳細:

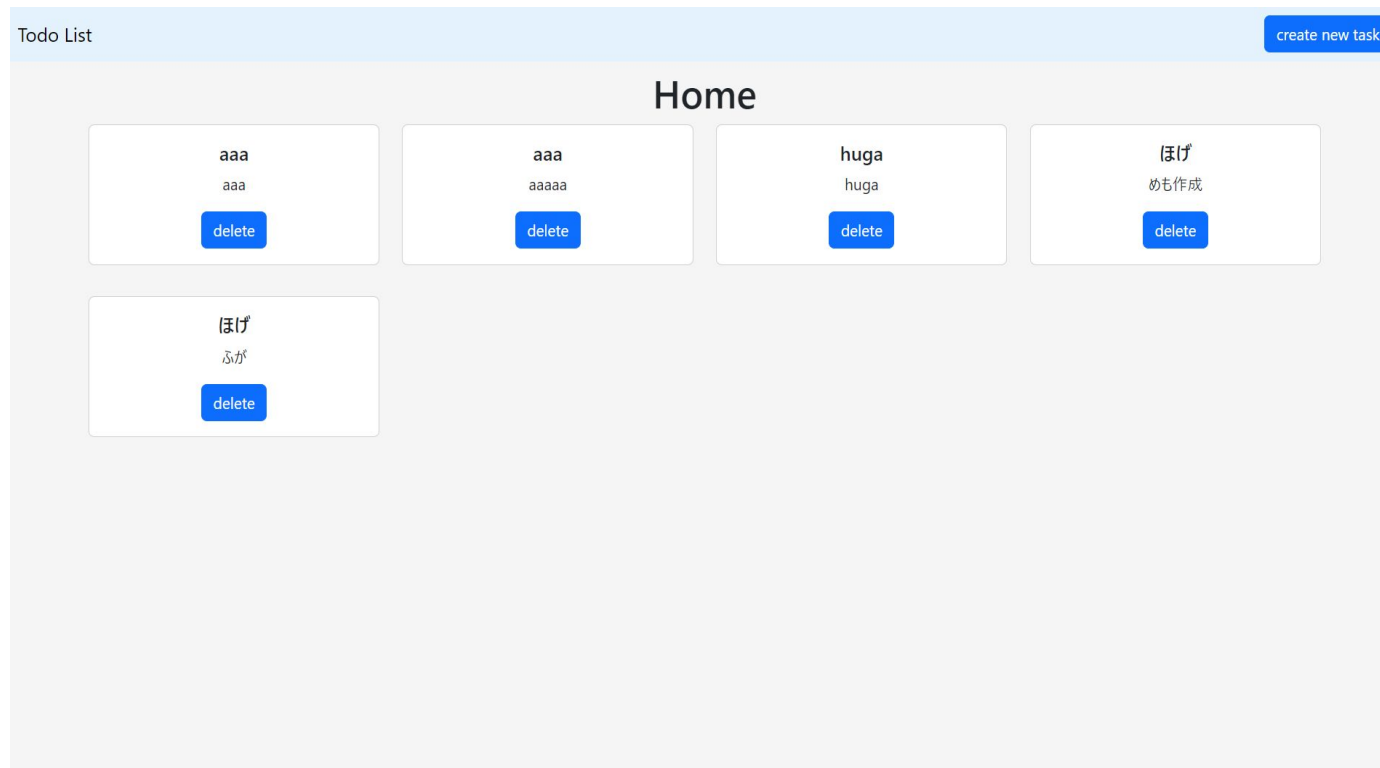
<メニュー>

・プッシュアップ

課題2の例

オプション課題

- アプリをおしゃれにしよう！



※例:これはBootstrapを使用していますが, 無理して使わなくていいです

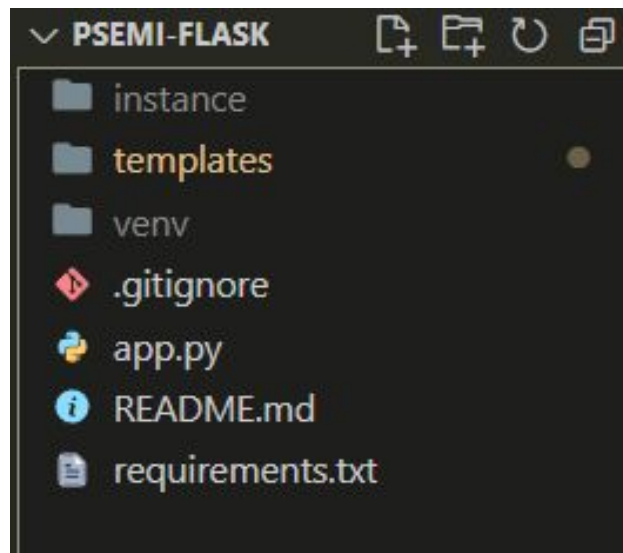
補足：CSSの書き方

CSSの書き方はこちらを参考にしてください.

https://willcloud.jp/knowhow/whats_css/#CSS-2

課題の前に

.gitignoreファイルを作成し、以下の内容を記載してください
記載したフォルダやファイルをGitが管理しなくなります
これまでのソースコードをGithubにpushしてください



```
instance/  
venv/
```

課題のやり方

1. 必須・オプション課題ごとにissueを作成します
2. issueに紐づいたブランチを作成します
3. 各ブランチで作業を行います
4. 完了したら, mainブランチにマージしてください

課題の締め切りは2025/12/18(木)